

LLVM & COMPILER ENGINEERING

LLVM ile Oyun Programlama Dili Geliştirme

Modern C++17/20, Frontend AST, LLVM IR, Pass Manager, CodeGen
& ORC JIT Rehberi

Geliştirici & Eğitim Kılavuzu
Kapsamlı Mimari ve Uygulama Kitabı

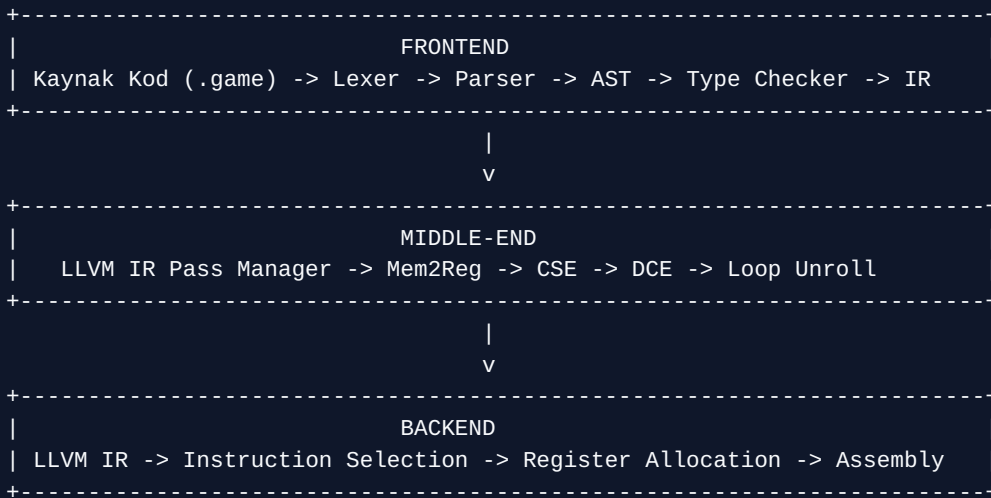
Bölüm 1: LLVM Mimarisine Giriş, Derleyici Mimarileri ve Oyun Motoru Dilleri İçin LLVM Ortam Kurulumu

1.1 Derleyici Mimarilerine Genel Bakış: Klasik vs. Modern Üç Aşamalı (Three-Phase) Yapı

Bir programlama dilinin kaynak kodunu (source code) hedef makinenin anlayabileceği makine koduna (machine code) dönüştürme süreci, bilgisayar bilimlerinin en karmaşık ve zarif alanlarından biridir. Geleneksel (monolitik) derleyicilerde, kaynak dilden hedef mimariye doğrudan bir geçiş yapılırdı. Bu durum, N adet programlama dili ve M adet hedef mimari (x86, ARM, RISC-V, WebAssembly vb.) olduğunda $N \times M$ adet farklı derleyici yazılmasını gerektiriyordu.

Modern derleyici tasarımında bu problem, **Üç Aşamalı Mimariler (Three-Phase Compiler Architecture)** ile çözülmüştür. Bu mimari üç ana bileşenden oluşur:

- 1. Frontend (Ön Yüz):** Kaynak kodu okur, sözdizimsel (lexical) ve dilbilgisel (syntactic) analizini yapar, Tip Denetimi (Type Checking) gerçekleştirir ve dili temsil eden bir **Soyut Sözdizim Ağacı (Abstract Syntax Tree - AST)** oluşturur. Son olarak bu AST'yi dile bağımsız bir **Ara Temsile (Intermediate Representation - IR)** dönüştürür.
- 2. Middle-end (Orta Yüz / Optimizasyon Katmanı):** Üretilen IR üzerinde çalışır. Kaynak dilden ve hedef donanımdan bağımsız olarak kod üzerindeki gereksiz hesaplamaları ayıklar, döngüleri optimize eder, bellek erişimlerini iyileştirir ve kodu en verimli hale getirir.
- 3. Backend (Arka Yüz / Kod Üretimi):** Optimize edilmiş IR'ı alır, hedef mimarinin yazmaç (register) kısıtlarına, buyruk kümesine (instruction set) ve yürütme boru hattına (pipeline) göre makine koduna (assembly / makine komutları) dönüştürür.



Bu üç aşamalı yaklaşım sayesinde N dil ve M mimari için ihtiyaç duyulan dönüşüm karmaşıklığı $N + M$ seviyesine inmiştir. LLVM, tam olarak bu mimarinin kalbinde yer alır.

Derleyici Mimarisinde Katman Ayrımı

Frontend dilden anlar fakat işlemciden anlamaz; Backend işlemciden anlar fakat dilden anlamaz; Middle-end ise ne dilden ne de işlemciden anlar, sadece evrensel LLVM IR üzerindeki matematiksel ve mantıksal optimizasyonlara odaklanır.

1.2 LLVM Felsefesi ve Ekosistemi

LLVM (kökensel olarak *Low Level Virtual Machine*, ancak günümüzde sadece bir marka isimdir), Chris Lattner ve Vikram Adve tarafından Illinois Üniversitesi'nde başlatılan açık kaynaklı bir derleyici altyapısı projesidir. LLVM'in temel felsefesi **Modülerlik**, **Yeniden Kullanılabilirlik** ve **Dil/Mimari Bağımsızlığıdır**.

LLVM Ekosistemi tek bir derleyiciden ibaret değildir; birbiriyle entegre çalışan devasa bir C++ kütüphaneleri ve araçlar koleksiyonudur:

- **LLVM Core:** Temel kod optimizasyonu ve her türlü işlemci mimarisi için kod üretimi sağlayan C++ kütüphaneleri koleksiyonu.
- **Clang:** C, C++, Objective-C ve OpenCL için geliştirilmiş, LLVM tabanlı ultra hızlı bir Frontend derleyicisi.
- **LLD:** LLVM projesinin geliştirdiği, GNU gold veya MSVC link.exe'ye kıyasla katlarca hızlı çalışan sistem bağlayıcısı (Linker).
- **LLDB:** LLVM mimarisi üzerine kurulu yüksek performanslı hata ayıklayıcı (Debugger).
- **MLIR (Multi-Level Intermediate Representation):** Etki alanına özel (Domain-Specific) derleyiciler ve yapay zeka/makine öğrenmesi modelleri için çok seviyeli ara temsil altyapısı.
- **LLVM ORC JIT:** Çalışma zamanında (JIT - Just-In-Time) dinamik kod derlemeyi sağlayan modüler C++ API'si.

1.3 Oyun Motorları İçin Özel Programlama Dili Geliştirme İhtiyacı

Oyun geliştirme; yüksek performans (FPS kısıtları), düşük gecikme (latency), esnek betik yazımı (scripting) ve GPU/CPU arasında kesintisiz haberleşme gerektiren zorlu bir alandır. C++ geleneksel olarak sektör standardı olsa da, derleme sürelerinin uzunluğu ve esnek olmayan yapısı nedeniyle oyun projelerinde sıklıkla betik dilleri (Lua, C#, Cscript vb.) kullanılır.

Ancak Lua gibi yorumlanan (interpreted) diller çalışma zamanında ciddi performans kayıplarına yol açar. Oyun motorları için özel bir dil oluştururken LLVM kullanmanın kritik avantajları şunlardır:

1. **Sıfır Maliyetli Soyutlama (Zero-Cost Abstraction):** LLVM'in güçlü optimizasyon katmanı sayesinde dilde yazdığınız yüksek seviyeli yapılar (örneğin 3D Vektör toplama, ECS bileşen erişimleri) doğrudan inline C++ seviyesinde makine koduna dönüşür.
2. **JIT Scripting Hibrit Yapısı:** Oyun oynanırken betik kodlarını çalışma zamanında LLVM ORC JIT ile makine koduna derleyebilir, interpret etme yavaşlığından tamamen kurtulabilirsiniz.
3. **GPU Shader ve CPU Kod Bütünlüğü:** LLVM SPIR-V veya DirectX Shader Compiler (DXC) altyapısı sayesinde oyun dilinizden doğrudan Vulkan/DirectX shader kodları üretebilirsiniz.
4. **Veri Odaklı Tasarım (Data-Oriented Design) Desteği:** Bellek düzenini (SIMD alignment, Struct-of-Arrays) LLVM IR seviyesinde milimetrik olarak kontrol edebilirsiniz.

1.4 Oyun Motoru C++ Geliştirme Ortamı ve LLVM Kurulumu

Oyun dilinizi yazarken LLVM C++ kütüphanelerini projenize bağlamanız gerekir. Modern C++ (C++17 / C++20) standartları ile LLVM API'lerini kullanacağız.

CMake Entegrasyonu (`CMakeLists.txt`)

Aşağıda, sisteminizde kurulu olan LLVM paketini otomatik tespit edip C++ projenize bağlayan profesyonel bir `CMakeLists.txt` konfigürasyonu yer almaktadır:

```
cmake_minimum_required(VERSION 3.20)
project(GameLangCompiler LANGUAGES CXX C)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# LLVM paketini sistemde ara
find_package(LLVM REQUIRED CONFIG)

message(STATUS "LLVM Bulundu: sürüm ${LLVM_PACKAGE_VERSION}")
message(STATUS "LLVM Dahil Etme Dizini: ${LLVM_INCLUDE_DIRS}")
message(STATUS "LLVM Tanımlamaları: ${LLVM_DEFINITIONS}")

# LLVM Başlık Dosyalarını Ekle
include_directories(${LLVM_INCLUDE_DIRS})
add_definitions(${LLVM_DEFINITIONS})

# Oyun Derleyicisi Çalıştırılabilir Dosyası
add_executable(gamelang_compiler
    src/main.cpp
)

# Gerekli LLVM Bileşenlerini/Kütüphanelerini Map'le
llvm_map_components_to_libnames(llvm_libs
    Core
    Analysis
    ExecutionEngine
    InstCombine
    Object
    OrcJIT
    ScalarOpts
    Support
    TransformUtils
    native
)

# Kütüphaneleri Hedefe Bağla
target_link_libraries(gamelang_compiler PRIVATE ${llvm_libs})
```

1.5 Derleyici Çekirdeği ve Çekirdek C++ Nesneleri: `LLVMContext` , `Module` , ve `IRBuilder` Adım Adım C++ Kodlaması

LLVM C++ API'sinde kod yazarken en çok karşılaşılabileceğiniz üç temel C++ yapısı bulunmaktadır. Bu yapılar sadece birer kütüphane nesnesi değil, derleyicinizin bellek ve yürütme mimarisinin omurgasını oluşturur. Bu

bölümde her bir nesnenin C++ tarafında adım adım nasıl oluşturulduğunu, değişkenlerin ne işe yaradığını ve arka planda kütüphanenin nasıl çalıştığını detaylıca işleyeceğiz.

1. `llvm::LLVMContext` : Derleyici Bellek Havuzu ve Tip Deposu

`LLVMContext` , LLVM'in tüm durum bilgilerini (state), tip tablolarını, sabit tanımlarını ve dahili veri yapılarını yöneten ana çekirdektir.

- **Çalışma Algoritması ve Mantığı:** LLVM bellek yönetimi performans odaklıdır. Örneğin `i32` (32-bit tamsayı) veya `float` tipini her ihtiyaç duyulduğunda yeniden `new` ile tahsis etmek yerine `LLVMContext` içerisinde Unification (Tekilleştirme / Flyweight Deseni) yöntemiyle tek bir defa oluşturur. Böylece milyonlarca değişkene sahip bir projede bile bellek kullanımı ve karşılaştırma (`==`) işlemleri pointer karşılaştırması hızına iner.
- **Benzetme:** `LLVMContext` nesnesini bir fabrikanın **Merkezi Deposu veya Hammaddede Omurgası** olarak düşünebilirsiniz. Fabrikada üretilen her ürün (talimatlar, tipler) bu depodaki hammaddeleri ortaklaşa kullanır.
- **İzlek Güvenliği (Thread Safety):** `LLVMContext` tek bir thread üzerinde çalışacak şekilde tasarlanmıştır. Çok izlekli (multithreaded) derleyicilerde (örneğin aynı anda 8 oyun betiğini paralel derlerken), her thread'in kendisine ait ayrı bir `LLVMContext` nesnesi olmalıdır.

C++ Tarafında Adım Adım Oluşturulması:

```
// 1. Adım: Standart std::unique_ptr ile bellek güvenliğini sağlayarak bir LLVMContext objesi tahsis edildi
// Bu 'context' objesi derleyici çalıştığı sürece tüm tiplerin ve sabitlerin tutulduğu merkezi veri deposu
std::unique_ptr<llvm::LLVMContext> context = std::make_unique<llvm::LLVMContext>();
```

2. `llvm::Module` : Derleme Birimi Konteyneri

`llvm::Module` , bir kaynak kod dosyasının (Translation Unit) LLVM dünyasındaki karşılığıdır.

- **İçerik ve Yapı:** Bir `Module` içerisinde fonksiyonlar (`llvm::Function`), küresel değişkenler (`llvm::GlobalVariable`), tip tanımları (`llvm::StructType`), veri düzeni (`llvm::DataLayout`) ve hedef donanım bilgisi (`Target Triple`) barındırır.
- **Benzetme:** `Module` yapısını bir oyun projesindeki **Tek Bir C++ Dosyası (.cpp) veya C# Class Dosyası** gibi düşünebilirsiniz. Tüm fonksiyonlar bu kutunun içerisinde ikamet eder.

C++ Tarafında Adım Adım Oluşturulması ve Bağlanması:

```
// 2. Adım: 'module' isimli derleme birimi konteyner objemizi oluşturuyoruz.
// İlk parametre modülün adıdır ("GameEngineMathModule").
// İkinci parametre ise modülün bağımlı olacağı bellek deposudur (*context).
// Bu bağlantı sayesinde modül içindeki tüm fonksiyon ve değişken tipleri bu context üzerinden yönetilecek
std::unique_ptr<llvm::Module> module = std::make_unique<llvm::Module>("GameEngineMathModule", *context);
```

3. `llvm::IRBuilder<>` : Kod Jeneratörü ve İmleç Yönetimi

`llvm::IRBuilder<>` , LLVM IR talimatlarını tür güvenli (type-safe) bir biçimde oluşturan ve bunları aktif Basic Block içerisine sırayla yazan yardımcı bir şablon sınıftır.

- **Çalışma Algoritması:** Builder nesnesi dahili bir **Ekleme Noktası İmleci (Insertion Point Cursor)** tutar. Siz `CreateFAdd` veya `CreateRet` çağırdığınızda, builder ilgili C++ talimat nesnesini imlecin gösterdiği Basic Block'un sonuna ekler ve imleci bir adım kaydırır.

- **Benzetme:** `IRBuilder` nesnesi bir kelime işlemci programındaki **Yazı Yazma İmleci (Text Cursor)** gibidir. İmleç neredeyse yeni yazılan metin (LLVM IR talimatı) tam oraya eklenir.

C++ Tarafında Adım Adım Oluşturulması ve Konumlandırılması:

```
// 3. Adım: IRBuilder sınıfından 'builder' adında bir kod üretici obje tanımlıyoruz.
// Yapıcı metoda (constructor) adres/referans olarak '*context' veriyoruz.
// 'builder' objesi, yazacağı IR komutlarının tiplerini ve hafıza yerleşimlerini doğrudan bu depodan çekebilir.
llvm::IRBuilder<> builder(*context);
```

Detaylı C++ Kod Anlatımı: Oyun Vektör Toplama Fonksiyonunun Adım Adım İnşası

(`src/main.cpp`)

Aşağıdaki C++ kodunda, bir oyun motorunun matematik kütüphanesinde yer alacak 2B Vektör Toplama fonksiyonunun (`vec2Add`) C++ LLVM API'si kullanılarak aşama aşama nasıl inşa edildiğini inceleyeceğiz.

```
#include <llvm/IR/LLVMContext.h>
#include <llvm/IR/Module.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/IR/Verifier.h>
#include <llvm/Support/raw_ostream.h>
#include <iostream>
#include <memory>
#include <vector>

int main() {
    // -----
    // ADIM 1: Merkezi Depo (LLVMContext) Objesinin Oluşturulması
    // -----
    // İlk olarak 'context' isimli akıllı göstericiyi (unique_ptr) tanımlıyoruz.
    // Bu obje, derleme sürecindeki tüm tip bilgilerini, dizgileri ve sabit değerleri
    // dahili tablosunda saklayan ana hafıza havuzudur.
    auto context = std::make_unique<llvm::LLVMContext>();

    // -----
    // ADIM 2: Derleme Birimi (Module) Objesinin Tanımlanması ve Bağlanması
    // -----
    // 'module' değişkenimiz, kaynak kod dosyamızın (örn. MathUtils.game) LLVM IR
    // karşılığı olan konteynerdir. 'context' referansını alarak onun hafıza
    // havuzuna doğrudan bağlanır.
    auto module = std::make_unique<llvm::Module>("GameEngineMathModule", *context);

    // -----
    // ADIM 3: Kod Yazıcı (IRBuilder) Objesinin Başlatılması
    // -----
    // 'builder' adında bir IRBuilder objesi üretiyoruz. Bu obje, komutları tek tek
    // oluşturup aktif temel bloğun (BasicBlock) içine imleç yardımıyla dizecektir.
    llvm::IRBuilder<> builder(*context);

    // -----
    // ADIM 4: Fonksiyon İmzası ve Tip Tanımlamalarının Yapılması
    // -----
    // Oyun dilimizde üretmek istediğimiz fonksiyon: float Vec2Add(float x1, float y1, float x2, float y2)
    // a) 'builder' üzerinden float veri tipini temsil eden 'floatType' pointer'ını çekiyoruz.
    llvm::Type* floatType = builder.getFloatTy();
```

```

// b) Fonksiyonun alacağı parametre tiplerini bir std::vector içinde topluyoruz.
// Dört adet kayan noktalı sayı (x1, y1, x2, y2) geçilecektir.
std::vector<llvm::Type*> paramTypes = {floatType, floatType, floatType, floatType};

// c) 'llvm::FunctionType::get' statik metodunu çağırarak fonksiyon tip şablonunu oluşturuyoruz.
// İlk parametre dönüş tipi (floatType), ikinci parametre girdi tipleri vektörü (paramTypes),
// üçüncü parametre ise variadic (değişken sayıda argüman) olup olmadığıdır (false).
llvm::FunctionType* funcType = llvm::FunctionType::get(floatType, paramTypes, false);

// d) 'llvm::Function::Create' metodu ile fonksiyon objemizi ('vec2AddFunc') oluşturuyoruz.
// Parametreler:
// 1. funcType: Fonksiyonun imzası
// 2. ExternalLinkage: Fonksiyonun dışarıdan (C/C++ tarafından) çağrılabilceğini belirtir.
// 3. "Vec2Add": Fonksiyonun sembol ismi.
// 4. module.get(): Fonksiyonun ekleneceği modül pointer'ı.
llvm::Function* vec2AddFunc = llvm::Function::Create(
    funcType,
    llvm::Function::ExternalLinkage,
    "Vec2Add",
    module.get()
);

// e) Fonksiyonun argüman isimlerini belirlemek için parametre iteratörünü başlatıyoruz.
// 'arg_begin()' ile başlayan iteratör üzerinde gezinerek her parametre objesine ad atıyoruz.
auto argIt = vec2AddFunc->arg_begin();
llvm::Value* x1 = argIt++; x1->setName("x1");
llvm::Value* y1 = argIt++; y1->setName("y1");
llvm::Value* x2 = argIt++; x2->setName("x2");
llvm::Value* y2 = argIt++; y2->setName("y2");

// -----
// ADIM 5: Temel Yapı Taşı (Basic Block) Oluşturma ve İmleç Bağlama
// -----
// a) 'entryBB' isimli ilk temel bloğumuzu oluşturuyoruz.
// Bu blok 'vec2AddFunc' fonksiyonunun giriş noktası olacaktır.
llvm::BasicBlock* entryBB = llvm::BasicBlock::Create(*context, "entry", vec2AddFunc);

// b) 'builder' kod yazıcı objemizin imlecini bu temel bloğun içine konumluyoruz ('SetInsertPoint').
// Artık 'builder' ile çağıracağımız tüm 'Create...' metodları komutları bu bloğa ekleyeceğiz.
builder.SetInsertPoint(entryBB);

// -----
// ADIM 6: Matematiksel Hesaplama Talimatlarının Üretilmesi
// -----
// Toplama işlemleri: x_sum = x1 + x2; y_sum = y1 + y2; total_sum = x_sum + y_sum

// a) 'builder.CreateFAdd' metodu iki float değerini toplayan LLVM IR 'fadd' komutunu üretir.
// Dönen 'xSum' objesi bir LLVM IR Sanal Yazmaca (Virtual Register) karşılık gelir.
llvm::Value* xSum = builder.CreateFAdd(x1, x2, "x_sum");

// b) Y bileşenlerinin toplamını üretiyoruz.
llvm::Value* ySum = builder.CreateFAdd(y1, y2, "y_sum");

// c) İki toplamı birleştirip nihai sonucu üretiyoruz.
llvm::Value* totalSum = builder.CreateFAdd(xSum, ySum, "total_sum");

// d) 'builder.CreateRet' metodu ile fonksiyonun sonlandırıcı talimatını (Terminator Instruction) yazıyoruz.
// 'totalSum' değerini çağırarak tarafa döndürür.
builder.CreateRet(totalSum);

// -----

```

```
// ADIM 7: Üretilen Kodun Yapısal Doğrulaması (Validation)
// -----
// 'llvm::verifyFunction' fonksiyonu oluşturduğumuz 'vec2AddFunc' C++ nesnesini tarayarak
// tip uyumsuzluğu veya eksik sonlandırıcı gibi bir kural ihlali olup olmadığını denetler.
if (llvm::verifyFunction(*vec2AddFunc, &llvm::outs())) {
    llvm::errs() << "HATA: Vec2Add fonksiyonu LLVM IR kurallarına uymuyor!\n";
    return 1;
}

// -----
// ADIM 8: Üretilen Metinsel LLVM IR Kodunun Ekrana Yazdırılması
// -----
llvm::outs() << "=== Oyun Dili Vec2Add LLVM IR Çıktısı ===\n";
module->print(llvm::outs(), nullptr);

return 0;
}
```

İpucu: LLVM IR Doğrulama (llvm::verifyFunction)

`llvm::verifyFunction` çağrısı, oluşturduğunuz IR'ın LLVM tip sistemine ve SSA kurallarına uyup uymadığını denetler. Derleyici geliştirirken bu adımı atlamamak, beklenmeyen çalışma zamanı çökmelerini önler.

Koda Adım Adım Derinlemesine Bakış ve Yürütme Algoritması

1. `auto context = std::make_unique<llvm::LLVMContext>();`
2. *Çalışma Mantığı:* LLVM evreninin ilk adımıdır. Tüm veri tipleri (float, i32, pointer vb.) bu context nesnesinin bellek tablosunda tutulur. Derleyicimiz sonlandığında `unique_ptr` sayesinde bellek sızıntısı olmadan otomatik temizlenir.
3. `auto module = std::make_unique<llvm::Module>("GameEngineMathModule", *context);`
4. *Çalışma Mantığı:* `GameEngineMathModule` isimli yeni bir derleme birimi konteyneri açar. Üretilcek `vec2Add` fonksiyonu bu modülün sembol tablosuna kaydedilecektir.
5. `llvm::FunctionType::get(floatType, paramTypes, false);`
6. *Çalışma Mantığı:* C++'taki `float(float, float, float, float)` fonksiyon imzasını oluşturur. Üçüncü parametre olan `false`, fonksiyonun variadic (değişken sayıda parametre alan `printf` gibi) olmadığını belirtir.
7. `llvm::Function::Create(...)`
8. *Çalışma Mantığı:* Fonksiyon nesnesini hafızada tahsis eder. `ExternalLinkage` parametresi, bu fonksiyonun C++ veya diğer dış kütüphaneler tarafından çağrılabilirliğini (public/global olduğunu) ifade eder.
9. `vec2AddFunc->arg_begin()` İteratörü
10. *Çalışma Mantığı:* Fonksiyonun parametre listesi üzerinde sırayla geçerek C++ nesnelere (`x1` , `y1` , `x2` , `y2`) isim atar. Bu isimler metin formatındaki `.ll` çıktısında değişken adı olarak görünür.
11. `BasicBlock::Create(*context, "entry", vec2AddFunc) ve builder.SetInsertPoint(entryBB)`
12. *Çalışma Mantığı:* Fonksiyonun ilk kod bloğunu (`entry`) açar. `SetInsertPoint` çağrısı `IRBuilder` imlecini bu bloğun içine konumlandırır. Artık builder ile yazılacak tüm komutlar bu bloğa eklenecektir.
13. `builder.CreateFAdd(...)`
14. *Çalışma Mantığı:* Kayan noktalı toplama (`fadd`) talimatlarını üretir. İlk çağrıda `x1` ve `x2` toplanıp sanal yazmaca yazılır, ikinci çağrıda `y1` ve `y2` toplanır, üçüncü çağrıda ise iki toplam birleştirilir.

15. `llvm::verifyFunction(*vec2AddFunc, ...)`

16. *Çalışma Mantığı*: Yapısal doğrulama algoritması çalıştırılır. Fonksiyonun bir sonlandırıcı (`ret`) ile bitip bitmediği, tiplerin uyuşup uyuşmadığı ve SSA kurallarının ihlal edilip edilmediği denetlenir.

1.6 Bölüm Özeti ve Derleyici Mimarisi Değerlendirmesi

Bu ilk bölümde, modern derleyici tasarımının temel taşı olan **Üç Aşamalı Mimariyi (Three-Phase Architecture)** ve bu mimarinin oyun programlama dili geliştirmedeki stratejik rolünü inceledik. $N \times M$ karmaşıklığını $N + M$ seviyesine indiren bu yapı, dilden ve donanımdan bağımsız evrensel bir ara temsil (LLVM IR) üzerinden çalışmaktadır.

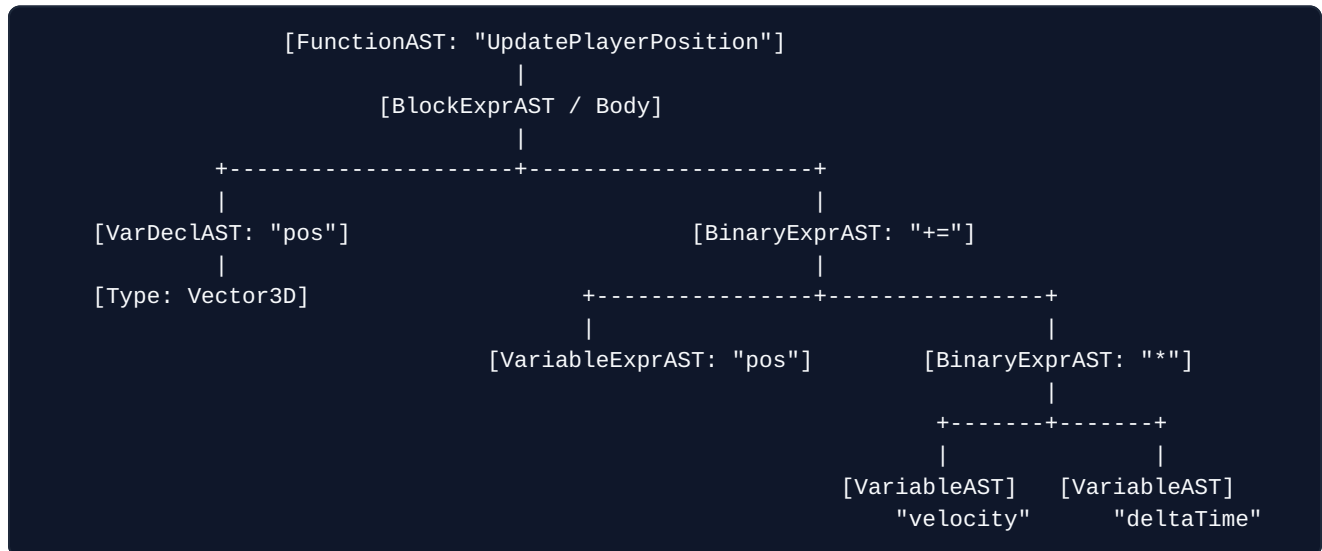
LLVM ekosisteminin sunduğu modüler C++ kütüphaneleri, Clang ön yüzü, LLD bağlayıcısı ve ORC JIT altyapısı sayesinde oyun motorlarında yüksek performans, canlı betik derleme (Hot-Reloading) ve verimli bellek yönetimi elde edilebilmektedir. C++ tarafında `LLVMContext` bellek deposu, `Module` derleme konteyneri ve `IRBuilder` kod üreticisi nesneleri ile ilk fonksiyonumuz olan `vec2Add` modülünü hafızada adım adım inşa edip doğruladık. Bir sonraki bölümde, dilimizin Ön Yüzünü (Frontend) inşa etmek üzere Soyut Sözdizim Ağaçları (AST), SSA formu ve LLVM bellek modelinin C++ ile adım adım uygulanmasını ele alacağız.

Bölüm 2: Frontend Tasarımı, AST, LLVM IR Yapısı ve Bellek Modeli

2.1 Soyut Sözdizim Ağacı (AST) Tasarımı

Ön Yüz (Frontend) mimarisinin kalbinde **Soyut Sözdizim Ağacı (Abstract Syntax Tree - AST)** yer alır. AST, kaynak kodun metinsel halindeki gereksiz karakterleri (parantezler, noktalı virgüller, boşluklar vb.) ayıklayarak kodun mantıksal ve dilbilgisel hiyerarşisini ağaç veri yapısı şeklinde temsil eder. Oyun programlama dilimizde hem ifade (expression) hem de bildirim/deyim (statement) düğümlerine ihtiyacımız vardır.

Modern C++ (C++17/20) standartlarında AST düğümlerini `std::unique_ptr` ve sanal fonksiyonlar (polymorphism) ile temsil ederiz. Her AST düğümü, kendini LLVM IR'a dönüştürme sorumluluğuna (`codegen()`) sahiptir.



2.2 LLVM IR (Intermediate Representation) Derinlemesine İnceleme

LLVM IR, LLVM derleyici altyapısının merkezindeki ortak dildir. Üç farklı biçimde temsil edilebilir: 1. **Hafızadaki (In-Memory) Veri Yapısı:** C++ sınıfları (`llvm::Instruction`, `llvm::Value`, `llvm::BasicBlock`). 2. **Disk Üzerindeki Okunabilir Metin Biçimi (.ll):** İnsanlar tarafından kolayca okunabilen montaj dili benzeri yapı. 3. **Disk Üzerindeki İkili Bitcode Biçimi (.bc):** Derleyicilerin hızlı işleyebilmesi için optimize edilmiş ikili format.

LLVM IR, güçlü bir şekilde tiplendirilmiş (strongly typed), sonsuz sayıda sanal yazmaca (virtual register) sahip ve **Tekli Atama Biçimine (Static Single Assignment - SSA)** dayalı bir yapıdır.

2.3 Tekli Atama Biçimi (Static Single Assignment - SSA) ve PHI Düşümleri

SSA Nedir ve Neden Hayatidir?

SSA formunda, her sanal yazmaca veya değişkene **yalnızca bir kez** değer atanabilir. Klasik bir programlama dilinde bir değişkene birden fazla kez değer atanabilir:

```
// Klasik C/C++ Kodu (Gereksiz Değişken Yeniden Atamaları)
int health = 100;
health = health - damage;
health = health + healAmount;
```

Bu kod SSA yapısına dönüştürüldüğünde, her atama yeni bir SSA sürüm değişkeni (register) oluşturur:

```
; LLVM IR SSA Biçimi
%health0 = copy i32 100
%health1 = sub i32 %health0, %damage
%health2 = add i32 %health1, %healAmount
```

- **Çalışma Algoritması ve Avantajı:** SSA formunun derleyicilere sağladığı en büyük avantaj, **Veri Akışı Analizini (Data Flow Analysis)** doğrusal hale getirmesidir. Bir yazmacın değerinin nereden geldiği ve nerede değiştiği tartışmasız bir şekilde bellidir. Derleyici, karmaşık gösterici (pointer) takibi yapmadan bağımlılıkları görür.
- **Benzetme:** SSA formunu bir muhasebe defterindeki **Silinemez Günlük Kayıtlara (Immutable Ledger / Blockchain)** benzetebiliriz. Eski bir girdinin üzerini çizip değiştiremezsiniz; her yeni durum için yeni bir satır kaydı açarsınız.

PHI Düşümleri (`phi` instruction) ve C++ ile Oluşturulması

Dallanma (if-else, döngüler) olan durumlarda bir değişkenin hangi kontrol akışı yolundan geldiği çalışma zamanında belli olur. SSA kuralını bozmadan bu durumu çözmek için **PHI Düşümleri (`phi` instruction)** kullanılır.

```
; Oyuncu Can Durumu Örneği LLVM IR Çıktısı
entry:
  %cmp = icmp sgt i32 %damage, %shield
  br i1 %cmp, label %take_damage, label %blocked

take_damage:
  %hp_after_hit = sub i32 %current_hp, %damage
  br label %merge

blocked:
  %hp_after_block = select i1 true, i32 %current_hp, i32 0
  br label %merge

merge:
  ; PHI Düşümü: Kodun hangi bloktan geldiğine göre doğru hp değerini seçer
  %final_hp = phi i32 [ %hp_after_hit, %take_damage ], [ %hp_after_block, %blocked ]
  ret i32 %final_hp
```

PHI Döğümünün C++ API'si ile Adım Adım Kodlanması:

C++ tarafında bir PHI döğümünü oluşturup bağlamak son derece nettir:

```
// 1. Adım: 'builder.CreatePHI' metodu ile PHI döğümü objemizi türetiyoruz.
// İlk parametre: Değişkenin tipi (32-bit tamsayı - i32).
// İkinci parametre: PHI döğümüne bağlanabilecek olası yol (incoming branches) sayısı (2 adet: take_damage, blocked).
// Üçüncü parametre: LLVM IR çıktısındaki sanal yazmaç adı ("final_hp").
llvm::PHINode* phiNode = builder.CreatePHI(builder.getInt32Ty(), 2, "final_hp");

// 2. Adım: 'phiNode' objemize '.addIncoming' metodunu kullanarak yollardan gelen değerleri bağlıyoruz.
// 'take_damage' bloğundan gelinmişse '%hp_after_hit' değerini seç diyoruz:
phiNode->addIncoming(phiAfterHitVal, takeDamageBB);

// 3. Adım: 'blocked' bloğundan gelinmişse '%hp_after_block' değerini seç diyoruz:
phiNode->addIncoming(phiAfterBlockVal, blockedBB);
```

PHI Döğümü Kuralı

LLVM IR içerisinde bir PHI döğümü talimatı (`phi`), bulunacağı Basic Block'un ****en başında**** (diğer tüm işlem talimatlarından önce) yer almak zorundadır!

2.4 Temel Yapı Taşları (Basic Blocks), Dallanmalar ve C++ İle Kontrol Akış Grafiğı (CFG) İnşası

Bir **Temel Yapı Taşı (Basic Block)**, içerisine yalnızca tek bir noktadan girilen (ilk talimat) ve yalnızca tek bir noktadan çıkılan (son talimat) talimatlar dizisidir.

Bir Basic Block'un son talimatı mutlaka bir **Terminator Instruction** (Sonlandırıcı Talimat) olmalıdır. Örnek sonlandırıcılar: `* ret` : Fonksiyondan döner. `* br` : Başka bir Basic Block'a dallanır (koşullu veya koşulsuz). `* switch` : Çoklu dallanma yapar.

C++ Tarafında Koşullu Dallanma (If-Else) ve CFG İnşası Adım Adım:

Aşağıdaki C++ kod parçası, oyuncunun canı 0'dan büyükse oyuncunun yaşamaya devam ettiği (`alive`), aksi halde öldüğü (`dead`) kontrol akış grafiğini (CFG) inşa eder:

```
// 1. Adım: Üç adet BasicBlock nesnesi oluşturuyoruz.
// Bunlar mantıksal dallanma rotalarımızdır.
llvm::BasicBlock* aliveBB = llvm::BasicBlock::Create(*context, "alive", playerFunc);
llvm::BasicBlock* deadBB = llvm::BasicBlock::Create(*context, "dead", playerFunc);
llvm::BasicBlock* mergeBB = llvm::BasicBlock::Create(*context, "merge", playerFunc);

// 2. Adım: Oyuncu canını (health) 0 ile karşılaştıran 'icmp sgt' (Signed Greater Than) komutunu üretiyoruz.
// Dönen 'condValue' objesi i1 (boolean: true/false) tipindedir.
llvm::Value* condValue = builder.CreateICmpSGT(healthVal, builder.getInt32(0), "is_alive");

// 3. Adım: 'builder.CreateCondBr' metodu ile koşullu dallanma talimatı üretiyoruz.
// Eğer 'condValue' true ise 'aliveBB' bloğuna, false ise 'deadBB' bloğuna atlar.
builder.CreateCondBr(condValue, aliveBB, deadBB);

// 4. Adım: 'aliveBB' bloğunun içine giriyoruz ve imlecimizi oraya konumlandırıyoruz.
builder.SetInsertPoint(aliveBB);
// Hayatta kalma mantığı komutları...
```

```
builder.CreateBr(mergeBB); // İş bitince ortak birleşme bloğuna (mergeBB) koşulsuz atla.

// 5. Adım: 'deadBB' bloğunun içine giriyoruz.
builder.SetInsertPoint(deadBB);
// Ölme mantığı komutları...
builder.CreateBr(mergeBB); // İş bitince ortak birleşme bloğuna (mergeBB) koşulsuz atla.

// 6. Adım: Son olarak imlecimizi 'mergeBB' birleşme noktasına getiriyoruz.
builder.SetInsertPoint(mergeBB);
```

2.5 Bellek Modeli, Yığın Tahsisi (`alloca`) ve `GetElementPtr` (GEP) Adres Hesaplama Algoritması

LLVM IR'da iki temel bellek erişim mantığı vardır: 1. **Yazmaç Tabanlı (Register-based SSA)**: `add`, `sub`, `mul` gibi işlemler doğrudan sanal yazmaçlar üzerinde yürür. 2. **Bellek Tabanlı (Memory-based Alloca)**: Yerel değişkenler yığında (`alloca`) tutulur, `load` ile okunur, `store` ile yazılır.

Yığın Tahsisi (`alloca`) ve C++ API Kullanımı

SSA formunu elle yönetmek zor olduğu için ön yüz derleyicileri genellikle tüm yerel değişkenleri fonksiyonun giriş bloğunda `alloca` ile oluşturur.

C++ Adım Adım `alloca` Oluşturulması:

```
// 1. Adım: Fonksiyonun giriş bloğunda 32-bit tamsayı türünde 'player_health' için yığın alanı ayarlıyoruz.
// Dönen 'healthAlloca' objesi bir Pointer (i32*) tipindedir.
llvm::AllocaInst* healthAlloca = builder.CreateAlloca(builder.getInt32Ty(), nullptr, "player_health");

// 2. Adım: 'builder.CreateStore' metodu ile yığındaki bu adrese ilk değer olarak 100 yazıyoruz.
builder.CreateStore(builder.getInt32(100), healthAlloca);

// 3. Adım: 'builder.CreateLoad' metodu ile yığındaki adresten değeri okuyup sanal bir yazmaçla yükleyelim.
llvm::Value* currentHealth = builder.CreateLoad(builder.getInt32Ty(), healthAlloca, "current_health_val");
```

LLVM'in `mem2reg` optimizasyon pass'i (Pass Manager), bu `alloca` bellek erişimlerini otomatik olarak SSA yazmaçlarına ve PHI düğümlerine dönüştürür!

`GetElementPtr` (GEP) Talimatı ve C++ ile Struct/Array Adres Hesaplaması

`GetElementPtr` (GEP), LLVM IR'ın en kritik ve sıkça yanlış anlaşılan talimatlarından biridir. GEP **belleğe erişmez (load/store yapmaz)**, yalnızca bellek adresini (pointer arithmetic) hesaplar!

Oyun motorumuzda bir `Entity` yapısının bellek düzenini düşünelim:

```
struct Entity {
    int id;           // 0. eleman (4 bayt)
    float health;     // 1. eleman (4 bayt)
    float position[3]; // 2. eleman (12 bayt: x, y, z)
};
```

C++ Tarafında Entity Yapısının C++ LLVM API ile Oluşturulması ve GEP Kullanımı:

```
// 1. Adım: C++ tarafında 'Entity' struct tipini LLVM'e tanıtıyoruz.
llvm::StructType* entityType = llvm::StructType::create(*context, "struct.Entity");

// Struct alan tipleri: i32 (id), float (health), [3 x float] (position dizisi)
llvm::ArrayType* posArrayType = llvm::ArrayType::get(builder.getFloatTy(), 3);
entityType->setBody({builder.getInt32Ty(), builder.getFloatTy(), posArrayType});

// 2. Adım: 'entityPtr' adında bir Entity* nesnesi olduğunu varsayalım (örn. fonksiyona parametre geldi).
// Oyuncunun position[1] (y koordinatı) adresini hesaplamak için GEP indekslerimizi hazırlıyoruz.
std::vector<llvm::Value*> indices = {
    builder.getInt32(0), // Indis 0: Pointer seviyesinde öteleme yok (entity_ptr[0])
    builder.getInt32(2), // Indis 2: Struct içindeki 2. alan (position dizisi)
    builder.getInt32(1)  // Indis 1: position dizisinin 1. elemanı (Y koordinatı)
};

// 3. Adım: 'builder.CreateGEP' metodu ile bellek adresi hesaplama talimatını üretiyoruz.
// İlk parametre struct tipi (entityType), ikinci parametre base pointer (entityPtr), üçüncü parametre indeksler.
llvm::Value* yCoordPtr = builder.CreateGEP(entityType, entityPtr, indices, "y_coord_ptr");

// 4. Adım: Artık elde ettiğimiz 'yCoordPtr' adresinden değeri 'load' ile okuyabiliriz:
llvm::Value* yValue = builder.CreateLoad(builder.getFloatTy(), yCoordPtr, "y_val");
```

- **Adres Hesaplama Formülü:** $\text{Hedef Adres} = \text{Taban Adres} + (0 \times \text{sizeof}(\text{Entity})) + \text{offsetof}(\text{Entity}, \text{position}) + (1 \times \text{sizeof}(\text{float}))$

GEP Neden İlk İndis Olarak '0' Alır?

C/C++ dilindeki `entity_ptr->position[1]` ifadesinde, `entity_ptr` aslında bir dizi nesnesinin ilk elemanının adresidir (`*(entity_ptr + 0)`). İlk indis olan '0', pointer seviyesinde kaç eleman öteye gidileceğini belirler.

2.6 IRBuilder<> Kullanımı ve AST'den LLVM IR Üretimi

Şimdi bu kavramları bir araya getirerek oyun dilimiz için tam teşekküllü bir AST yapısı ve bu AST'yi C++ LLVM API'si kullanarak adım adım LLVM IR'a dönüştüren derleyici kodunu yazalım.

C++ Örneği: Tam Teşekküllü AST ve IR Generation (`src/frontend_ast.cpp`)

```
#include <llvm/IR/LLVMContext.h>
#include <llvm/IR/Module.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/IR/Value.h>
#include <llvm/IR/Verifier.h>
#include <llvm/Support/raw_ostream.h>
#include <memory>
#include <string>
#include <vector>
#include <map>

// Derleyici Sembol Tablosu (Symbol Table): Değişken Adı -> Yığın Adresi (AllocaInst*)
static std::map<std::string, llvm::AllocaInst*> NamedValues;

// Taban AST Sınıfı
class ExprAST {
public:
```

```

    virtual ~ExprAST() = default;
    virtual llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) const;
};

// Sayısal Sabit AST Düzümü (Örn: 10.0, 2.5)
class NumberExprAST : public ExprAST {
    double val;
public:
    NumberExprAST(double val) : val(val) {}

    llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) const {
        // ConstantFP::get metodu ile kütüphanemizin merkezi deposundan float sabiti alıyoruz.
        return llvm::ConstantFP::get(context, llvm::APFloat(val));
    }
};

// Değişken Okuma AST Düzümü (Örn: playerHealth)
class VariableExprAST : public ExprAST {
    std::string name;
public:
    VariableExprAST(const std::string& name) : name(name) {}

    llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) const {
        // 1. Adım: Değişken adını sembol tablomuzda (NamedValues) arıyoruz.
        llvm::AllocaInst* alloca = NamedValues[name];
        if (!alloca) {
            llvm::errs() << "HATA: Tanımsız değişken kullanımı: " << name << "\n";
            return nullptr;
        }
        // 2. Adım: Değişken yığında bulunduğu için 'CreateLoad' ile adresindeki değeri okuyup döndürüyoruz.
        return builder.CreateLoad(alloca->getAllocatedType(), alloca, name.c_str());
    }
};

// İkili İşlem (Binary Expression: +, -, *) AST Düzümü
class BinaryExprAST : public ExprAST {
    char op;
    std::unique_ptr<ExprAST> lhs, rhs;
public:
    BinaryExprAST(char op, std::unique_ptr<ExprAST> lhs, std::unique_ptr<ExprAST> rhs)
        : op(op), lhs(std::move(lhs)), rhs(std::move(rhs)) {}

    llvm::Value* codegen(llvm::LLVMContext& context, llvm::Module& module, llvm::IRBuilder<>& builder) const {
        // Özyineli olarak sol ve sağ alt ağaçların IR kodlarını üretiyoruz.
        llvm::Value* l = lhs->codegen(context, module, builder);
        llvm::Value* r = rhs->codegen(context, module, builder);
        if (!l || !r) return nullptr;

        // İşlem operatörüne göre ilgili LLVM IR talimatını üretiyoruz.
        switch (op) {
            case '+': return builder.CreateFAdd(l, r, "addtmp");
            case '-': return builder.CreateFSub(l, r, "subtmp");
            case '*': return builder.CreateFMul(l, r, "multmp");
            default: return nullptr;
        }
    }
};

// Kullanım Örneği Fonksiyonu
void generateSampleAST() {
    // 1. LLVM Çekirdek Obje Kurulumları

```

```

auto context = std::make_unique<llvm::LLVMContext>();
auto module = std::make_unique<llvm::Module>("GameASTModule", *context);
llvm::IRBuilder<> builder(*context);

// 2. Örnek İfade Ağacı Oluşturma: (10.0 + 20.0) * 2.5 (Oyun içi Hasar Çarpanı Hesabı)
auto expr = std::make_unique<BinaryExprAST>(
    '*',
    std::make_unique<BinaryExprAST>('+', std::make_unique<NumberExprAST>(10.0), std::make_unique<NumberExprAST>(20.0)),
    std::make_unique<NumberExprAST>(2.5)
);

// 3. Fonksiyon İmzası Oluşturma: double CalculateDamage()
llvm::FunctionType* ft = llvm::FunctionType::get(builder.getDoubleTy(), false);
llvm::Function* func = llvm::Function::Create(ft, llvm::Function::ExternalLinkage, "CalculateDamage", *context);
llvm::BasicBlock* bb = llvm::BasicBlock::Create(*context, "entry", func);
builder.SetInsertPoint(bb);

// 4. AST Kod Üretimini Tetikleme
llvm::Value* result = expr->codegen(*context, *module, builder);
builder.CreateRet(result);

// 5. Doğrulama ve Çıktı
llvm::verifyFunction(*func);
module->print(llvm::outs(), nullptr);
}

```

Koda Adım Adım Derinlemesine Bakış ve Yürütme Algoritması

1. **ExprAST::codegen** Sanal Fonksiyon Özyinelemesi (Recursion):
2. **Çalışma Mantığı:** Ağacın en altındaki (leaf nodes) sabitler (NumberExprAST) önce ziyaret edilir. Alt düğümlerden dönen `llvm::Value*` göstericileri üst düğümlere (BinaryExprAST) girdi olarak iletilir. Bu yöntem Post-Order Traversal (Önce Sol, Sonra Sağ, Sonra Kök) algoritmasıdır.
3. **builder.CreateFAdd / CreateFMul :**
4. **Çalışma Mantığı:** İki float değerini toplayan veya çarpan LLVM IR talimatlarını oluşturur ve `addtmp / multmp` isimli sanal yazmaçlara atar.
5. **builder.CreateRet(result) :**
6. **Çalışma Mantığı:** Bloğun sonlandırıcı talimatını (Terminator) ekler ve hesaplanan nihai değeri fonksiyondan döndürür.

2.7 Bölüm Özeti ve Ön Yüz Mimarisi Değerlendirmesi

Bu bölümde, oyun programlama dilimizin Ön Yüz (Frontend) mimarisini baştan sona C++ LLVM API'lerini kullanarak ele aldık. Metinsel kaynak kodun **Soyut Sözdizim Ağacına (AST)** dönüştürülmesini, özyineli `codegen()` mimarisi ile bu ağacın LLVM IR'a aktarılmasını inceledik.

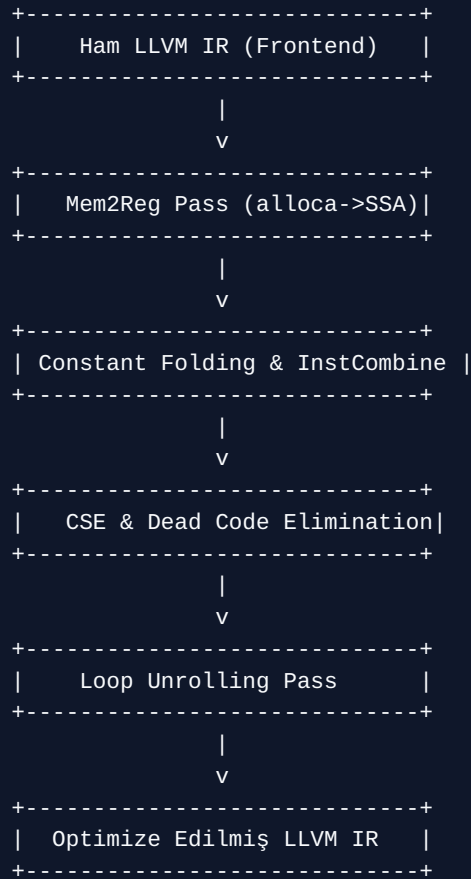
LLVM IR'ın bellek ve veri akışı omurgasını oluşturan **Static Single Assignment (SSA)** formunu, dallanmalardaki değer belirsizliklerini çözen **PHI Düğümlerini (phi)**, `CreateCondBr` ile kontrol akış grafiklerini (CFG) ve yığın tahsisi (`alloca`) ile bellek adres hesaplaması yapan `GetElementPtr` (**GEP**) talimatının C++ API ile adım adım kodlanmasını kavradık. Bir sonraki bölümde, ürettiğimiz bu ham LLVM IR'ı optimize etmek üzere Middle-End katmanına ve **New Pass Manager (NPM)** altyapısına geçeceğiz.

Bölüm 3: Middle-end Optimizasyonları ve New Pass Manager (NPM)

3.1 LLVM Optimizasyon Mimarisi ve Pass Kavramı

LLVM derleyici altyapısının en güçlü yönlerinden biri modüler **Pass (Geçiş)** mimarisidir. Bir **Pass**, LLVM IR üzerinde belirli bir analizi (Analysis Pass) yürüten veya IR'ı değiştiren/optimize eden (Transformation Pass) bağımsız bir C++ sınıfıdır.

Middle-end optimizasyon katmanı, kaynak dilden ve hedef donanımdan tamamen bağımsızdır. Amacı, ön yüzün ürettiği LLVM IR'ı matematiksel ve mantıksal olarak en sade, en hızlı ve en az bellek tüketen şekle getirmektir.



3.2 New Pass Manager (NPM) vs. Legacy Pass Manager

LLVM 13 ve sonrasında eski (Legacy) Pass Manager tamamen kaldırılmış ve yerini **New Pass Manager (NPM)** almıştır. New Pass Manager'ın getirdiği temel yenilikler ve avantajlar şunlardır:

1. **Tür Güvenliği ve Derleme Zamanı Şablonları (Templates & CRTP):** Kalıtım (vtable) yerine C++ şablonları ve Mixin yapısı (`llvm::PassInfoMixin`) kullanılarak sanal fonksiyon çağrı maliyetleri sıfırlanmıştır.

2. **Esnek Analiz Önbellekleme (Analysis Management):** Bir optimizasyon pass'i IR'ı değiştirmediyse, önceki analiz sonuçları (örn: Dominator Tree, Loop Info) tekrar hesaplanmaz (`PreservedAnalyses::all()`).
3. **Piyasa Standartları Pipeline Desteği:** `01` , `02` , `03` , `0s` , `0z` gibi hazır optimizasyon seviyeleri tek hat üzerinden yapılandırılabilir.

NPM vs Legacy Performans Farkı

New Pass Manager, şablon tabanlı yapısı ve akıllı analiz önbellekleme (Analysis Caching) mekanizması sayesinde büyük projelerde derleme sürelerini %10 ile %25 arasında kısaltmaktadır.

3.3 Temel Optimizasyon Teknikleri ve IR Üzerinde İncelenmesi

Oyun motorlarında en yüksek kare hızlarına (FPS) ulaşmak için kritik öneme sahip 4 ana optimizasyon tekniğini inceleyelim:

1. Sabit Katlama (Constant Folding)

Derleme zamanında değeri bilinen sabit ifadelerin hesaplanarak tek bir değere indirgenmesidir.

• Optimize Edilmemiş IR:

```
%a = fadd float 10.0, 20.0
%b = fmul float %a, 2.0
```

• Constant Folding Sonrası IR:

```
; Derleyici (10.0 + 20.0) * 2.0 = 60.0 işlemini derleme zamanında yapar!
%b = store float 60.0
```

2. Ortak Alt İfade Eleme (Common Subexpression Elimination - CSE)

Program içinde birden fazla kez tekrarlanan aynı matematiksel hesaplamaları tespit edip tek bir hesaplama sonucunu tekrar kullanmaktır. Oyun içi vektör uzaklık hesaplarında hayati önem taşır.

• Ham IR:

```
%dx1 = fsub float %x2, %x1
%dy1 = fsub float %y2, %y1
; Tekrar eden aynı ifade:
%dx2 = fsub float %x2, %x1
```

• CSE Sonrası IR:

```
%dx1 = fsub float %x2, %x1
%dy1 = fsub float %y2, %y1
; %dx2 ifadesi silindi, %dx1 yazmacı doğrudan kullanıldı
```

3. Ölü Kod Eleme (Dead Code Elimination - DCE)

Programın çıktısını veya durumunu etkilemeyen, sonucu hiçbir yerde kullanılmayan (unreachable / unused) talimatların silinmesidir.

- Ham IR:

```
%unused_calc = fmul float %enemy_hp, 0.0 ; Sonucu hiçbir yerde kullanılmıyor
ret void
```

- DCE Sonrası IR:

```
ret void ; %unused_calc tamamen temizlendi!
```

4. Döngü Açma (Loop Unrolling)

Döngü dallanma maliyetini (branch prediction ve sayaç artırma) azaltmak için döngü gövdesini ardışık olarak genişletmektir. Oyun motorlarında matris çarpmalarında ve SIMD vektörizasyonunda sıklıkla uygulanır.

3.4 Özel Bir LLVM Pass Yazımı (Modern C++ NPM)

Şimdi oyun derleyicimiz için custom bir LLVM Pass yazalım. Bu Pass, oyun kodumuzdaki tüm yavaş bölme işlemlerini (fdiv) çarpmaya (fmul) dönüştüren bir optimizasyon yapsın.

C++ Kodu (src/custom_pass.cpp)

```
#include <llvm/IR/PassManager.h>
#include <llvm/IR/Instructions.h>
#include <llvm/IR/Constants.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/Support/raw_ostream.h>

// New Pass Manager için Custom Pass Sınıfı
class GameFastMathPass : public llvm::PassInfoMixin<GameFastMathPass> {
public:
    llvm::PreservedAnalyses run(llvm::Function& F, llvm::FunctionAnalysisManager& FAM) {
        bool changed = false;
        llvm::outs() << "GameFastMathPass Calisiyor: Fonksiyon " << F.getName() << "\n";

        for (auto& BB : F) {
            for (auto InstIt = BB.begin(); InstIt != BB.end(); ) {
                llvm::Instruction& I = *InstIt++;

                // Eğer bir FDiv (Float Bölme) işlemiyse ve bölen sabit bir sayıysa
                if (auto* fdiv = llvm::dyn_cast<llvm::BinaryOperator>(&I)) {
                    if (fdiv->getOpcode() == llvm::Instruction::FDiv) {
                        if (auto* c = llvm::dyn_cast<llvm::ConstantFP>(fdiv->getOperand(1))) {
                            // 1.0 / Constant hesabı derleme zamanında yap
                            double val = c->getValueAPF().convertToDouble();
                            if (val != 0.0) {
                                double recip = 1.0 / val;
                                llvm::IRBuilder<> builder(fdiv);
                                llvm::Value* recipVal = builder.getFloatTy()->isFloatTy() ?
                                    (llvm::Value*)llvm::ConstantFP::get(builder.getFloatTy(), recip) :
```

```

        (llvm::Value*)llvm::ConstantFP::get(builder.getDoubleTy(), recip);

        // FDiv yerine FMul oluştur
        llvm::Value* newMul = builder.CreateFMul(fdiv-&gtgetOperand(0), recipval, "newMul");
        fdiv-&gtreplaceAllUsesWith(newMul);
        fdiv-&gteraseFromParent();
        changed = true;
    }
}
}
}
}

// Eğer IR değiştiyse analizleri invalidate et, değişmediyse koru
return changed ? llvm::PreservedAnalyses::none() : llvm::PreservedAnalyses::all();
}
};

```

İpucu: eraseFromParent() Kullanımı

Bir LLVM talimatını silmeden önce (`eraseFromParent()`), onun ürettiği sonucu kullanan diğer tüm talimatların `replaceAllUsesWith()` ile yeni talimata yönlendirildiğinden emin olunmalıdır!

Koda Adım Adım Derinlemesine Bakış ve Pass Yürütme Algoritması

1. **PassInfoMixin<GameFastMathPass> Kalıtımı (CRTP Pattern):**
2. **Çalışma Mantığı:** Curiously Recurring Template Pattern (CRTP) yapısıdır. Sanal fonksiyon tablosu (vtable) masrafı olmadan static polymorphism sağlar.
3. **run(Function& F, FunctionAnalysisManager& FAM) Metodu:**
4. **Çalışma Mantığı:** NPM her fonksiyon için bu metodu otomatik çağırır. `FAM` parametresi sayesinde Dominator Tree veya Loop Info gibi analizler hazır alınabilir.
5. **Güvenli İteratör Döngüsü (InstIt++):**
6. **Çalışma Mantığı:** Döngü içinde bir talimat silineceği için (`eraseFromParent()`), iteratör önceden artırılır (`*InstIt++`). Aksi takdirde dangling pointer hatası ile derleyici çöker!
7. **replaceAllUsesWith & eraseFromParent :**
8. **Çalışma Mantığı:** Eski bölme talimatına bağlı olan tüm SSA tüketicilerini yeni çarpma talimatına yönlendirir ve eski talimatı bellekten siler.
9. **PreservedAnalyses Dönüş Değeri:**
10. **Çalışma Mantığı:** Eğer fonksiyonda değişiklik yapıldıysa `none()` dönerek diğer analizlerin tekrar hesaplanmasını sağlar; değişiklik yoksa `all()` dönerek ön belleği korur.

3.5 Pass Pipeline Oluşturma ve Optimizasyon Çalıştırma

Oyun derleyicimizde bu custom pass'ı ve LLVM'in standart optimizasyon pipeline'ını nasıl çalıştıracağımızı görelim:

```

#include <llvm/Passes/PassBuilder.h>
#include <llvm/IR/PassManager.h>

```

```

void runOptimizationPipeline(llvm::Module& module) {
    llvm::LoopAnalysisManager LAM;
    llvm::FunctionAnalysisManager FAM;
    llvm::CGSCCAnalysisManager CGAM;
    llvm::ModuleAnalysisManager MAM;

    llvm::PassBuilder PB;

    // Analiz yöneticilerini kaydet
    PB.registerModuleAnalyses(MAM);
    PB.registerFunctionAnalyses(FAM);
    PB.registerRegisterClassBuilders(CGAM, LAM, FAM, MAM);
    PB.crossRegisterProxies(LAM, FAM, CGAM, MAM);

    // Module Pass Manager
    llvm::ModulePassManager MPM;

    // Standart O2 Optimizasyon Pipeline'ını Yükle
    MPM = PB.buildPerModuleDefaultPipeline(llvm::OptimizationLevel::O2);

    // Pipeline'ı Modül Üzerinde Çalıştır
    MPM.run(module, MAM);
    llvm::outs() << "=== Modul Optimizasyonlari Tamamlandi ===\n";
}

```

3.6 Bölüm Özeti ve Orta Yüz (Middle-end) Optimizasyon Değerlendirmesi

Bu bölümde, derleyicimizin Orta Yüz (Middle-End) katmanını ve **New Pass Manager (NPM)** mimarisini detaylıca öğrendik. C++ şablonları tabanlı NPM yapısının eski Legacy Pass Manager'a göre sağladığı hız ve analiz ön bellekleme üstünlüklerini inceledik.

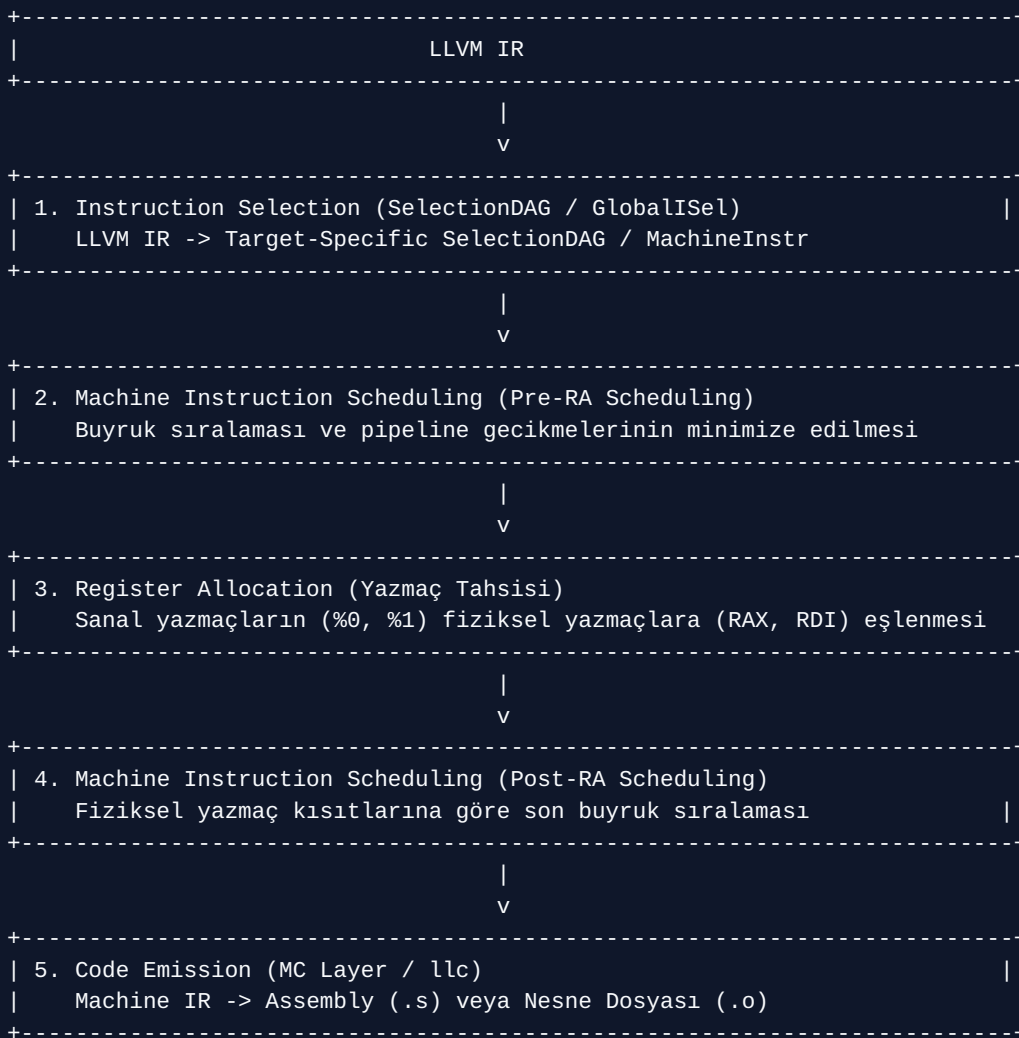
Oyun motorlarının yüksek başarımlı gereksinimleri için hayati önem taşıyan Sabit Katlama (Constant Folding), Ortak Alt İfade Eleme (CSE), Ölü Kod Eleme (DCE) ve Döngü Açma (Loop Unrolling) tekniklerinin LLVM IR üzerindeki dönüşümlerini izledik. Yavaş bölme işlemlerini çarpmaya çeviren özgün bir `GameFastMathPass` C++ sınıfı kodlayarak analiz ve dönüştürme pass'lerinin çalışma algoritmalarını kavradık. Bir sonraki bölümde, optimize edilmiş bu IR'ı gerçek donanım komutlarına (Assembly / Object File) dönüştürmek üzere **Backend Kod Üretimi ve Target Machine** mimarisine geçeceğiz.

Bölüm 4: Backend Kod Üretimi, CodeGen Mimarisi ve Hedef Makine (Target Machine)

4.1 LLVM Backend Boru Hattı (CodeGen Pipeline)

Middle-end katmanı tarafından optimize edilmiş LLVM IR, **Backend (Arka Yüz)** katmanına girdi olarak verilir. Backend'in görevi, bu soyut IR'ı hedef donanımın (x86_64, ARM64, RISC-V, WebAssembly vb.) mimarisine özel makine komutlarına ve ikili dosyalara (.o / .obj) dönüştürmektir.

LLVM Backend boru hattı 5 ana evreden oluşur:



Backend Esnekliği

Geliştirdiğiniz oyun dili için tek bir satır dahi C++ backend kodu yazmadan, sadece LLVM IR üreterek x86_64, ARM64 (Apple Silicon, Android), WebAssembly ve RISC-V mimarilerine kod çıktısı alabilirsiniz!

4.2 TableGen (.td) Dili ve Donanım Tanımlama

LLVM Backend'lerin en yenilikçi taraflarından biri **TableGen** altyapısıdır. Hedef işlemcinin yazmaçları, komut seti (instruction set), adresleme modları ve boru hattı (pipeline) özellikleri C++ koda elle sabitlenmek yerine `.td` uzantılı TableGen dosyalarında beyan edilir.

`llvm-tblgen` aracı bu `.td` dosyalarını okuyarak backend derlemesi sırasında otomatik C++ kodları üretir.

Örnek TableGen Tanımı (`TargetRegister.td` konsepti)

```
// x86_64 Akümülatör Yazmacı Tanımı
def RAX : Register<"rax">;
def RBX : Register<"rbx">;

// Donanım Komutu Tanımı: ADD komutu
class x86_instruction<bits<8> opcode, string asmstr> : Instruction {
  let OutOperandList = (outs GR64:$dst);
  let InOperandList = (ins GR64:$src1, GR64:$src2);
  let AsmString = asmstr;
}

def ADD64rr : x86_instruction<0x01, "addq $src2, $dst">;
```

4.3 Buyruk Seçimi (Instruction Selection): SelectionDAG vs. GlobalSel

Buyruk Seçimi, LLVM IR talimatlarını hedef işlemcinin desteklediği somut makine komutlarıyla eşleme işlemidir.

1. **SelectionDAG (Geleneksel Yöntem):**
2. LLVM IR'ı yönlü devirsiz grafiklere (Directed Acyclic Graph - DAG) dönüştürür.
3. Düğümleri donanım komut modelleriyle eşleştirir (Pattern Matching).
4. Yüksek optimizasyonlu kod üretir ancak hafıza kullanımı fazla ve derleme süresi daha uzundur.
5. **GlobalSel (Global Instruction Selection - Modern Yöntem):**
6. Fonksiyonun tamamı üzerinde DAG oluşturmada Doğrudan Machine IR (MIR) seviyesinde çalışır.
7. Hızlı derleme süreleri sağlar (JIT ve debug derlemeleri için idealdir).

4.4 Yazmaç Tahsisi (Register Allocation) ve Scheduling

LLVM IR sonsuz sayıda **sanal yazmaç (virtual register)** kullanabilir (`%1`, `%2`, `%3` ...). Ancak gerçek işlemcilerin kısıtlı sayıda **fiziksel yazmacı (physical register)** vardır (örn: x86_64 için 16 genel amaçlı yazmaç).

Register Allocation (RA)

Register Allocator, sanal yazmaçları fiziki yazmaçlara atar. Eğer fiziki yazmaç sayısı yetersiz kalırsa, bazı değerleri yığına saklar ve tekrar okur. Bu duruma **Spilling** adı verilir.

Instruction Scheduling

İşlemci boru hattında (pipeline stalls) veri bağımlılıklarından kaynaklanan beklemleri önlemek için komutların sırasını değiştirir.

Spilling Maliyeti

Register Spilling, işlemci yazmacı yerine RAM/L1 Cache erişimi gerektirdiği için oyun gibi sıkı performans gerektiren döngülerde yavaşlamaya neden olur. LLVM Register Allocator bunu minimize edecek algoritmalar (Greedy Register Allocator) kullanır.

4.5 Machine IR (MIR) ve `llc` Aracı

Machine IR (MIR), LLVM IR'ın hedef mimariye özgü komutlarla temsil edilmiş halidir. `.mir` uzantılı dosyalar halinde diske aktarılabilir ve `llc` (LLVM Static Compiler) aracı ile doğrudan incelenebilir.

```
# LLVM IR dosyasını x86_64 Assembly koduna dönüştürme
llc -march=x86-64 -O3 input.ll -o output.s

# LLVM IR dosyasını doğrudan Nesne Dosyasına (.o) dönüştürme
llc -filetype=obj -march=x86-64 input.ll -o output.o
```

4.6 C++ API'si İle TargetMachine Yapılandırması ve Nesne Kod Üretimi (.o)

Oyun derleyicimizin en son aşamasında, ürettiğimiz LLVM modülünü diskte `.o` (Object File) olarak kaydedecek C++ kodunu yazalım.

C++ Kodu (`src/backend_codegen.cpp`)

```
#include <llvm/IR/Module.h>
#include <llvm/Support/TargetSelect.h>
#include <llvm/Support/TargetRegistry.h>
#include <llvm/Target/TargetMachine.h>
#include <llvm/Target/TargetOptions.h>
#include <llvm/Support/FileSystem.h>
#include <llvm/Support/raw_ostream.h>
#include <llvm/MC/TargetRegistry.h>
#include <llvm/IR/LegacyPassManager.h>
#include <system_error>

bool emitObjectFile(llvm::Module& module, const std::string& outputFilename) {
    // 1. Tüm Hedef Mimarileri İlklendir
    llvm::InitializeAllTargetInfos();
    llvm::InitializeAllTargets();
    llvm::InitializeAllTargetMCs();
    llvm::InitializeAllAsmParsers();
    llvm::InitializeAllAsmPrinters();

    // 2. Varsayılan Hedef Üçlüsünü (Target Triple) Al (Örn: x86_64-pc-linux-gnu)
    auto targetTriple = llvm::sys::getDefaultTargetTriple();
    module.setTargetTriple(targetTriple);
```



```

std::string error;
auto target = llvm::TargetRegistry::lookupTarget(targetTriple, error);

if (!target) {
    llvm::errs() << "HATA: Target bulunamadi: " << error << "\n";
    return false;
}

// 3. Hedef Makine Yapılandırması (CPU: generic, Features: none)
auto cpu = "generic";
auto features = "";

llvm::TargetOptions opt;
auto rm = std::optional<llvm::Reloc::Model>();
auto targetMachine = target->createTargetMachine(targetTriple, cpu, features, opt, rm);

module.setDataLayout(targetMachine->createDataLayout());

// 4. Çıktı Dosyasını Açma (.o)
std::error_code ec;
llvm::raw_fd_ostream dest(outputFilename, ec, llvm::sys::fs::OF_None);

if (ec) {
    llvm::errs() << "HATA: Çıktı dosyası açilamadı: " << ec.message() << "\n";
    return false;
}

// 5. CodeGen Pass Manager ile Nesne Dosyası Üretimi
llvm::legacy::PassManager pass;
auto fileType = llvm::CodeGenFileType::CGFT_ObjectFile;

if (targetMachine->addPassesToEmitFile(pass, dest, nullptr, fileType)) {
    llvm::errs() << "HATA: TargetMachine bu dosya tipini üretmiyor!\n";
    return false;
}

pass.run(module);
dest.flush();

llvm::outs() << "BAŞARILI: Nesne dosyası üretildi -> " << outputFilename << "\n";
return true;
}

```

Koda Adım Adım Derinlemesine Bakış ve Yürütme Algoritması

1. **Target Sürücülerinin Kaydı (`InitializeAllTargets vb.`):**
2. **Çalışma Mantığı:** Statik LLVM kütüphanelerindeki tüm mimari sürücülerini (x86, ARM, WebAssembly vb.) runtime kayıt tablosuna ekler.
3. **`TargetRegistry::lookupTarget` :**
4. **Çalışma Mantığı:** Sistem hedef üçlüsünü (Triple) inceleyerek ilgili hedef mimarinin derleyici fabrika nesnesini arar ve bulur.
5. **`createTargetMachine` ve `createDataLayout` :**
6. **Çalışma Mantığı:** Hedef donanımın bellek hizalama kurallarını (Data Layout: little-endian, pointer boyutu 64-bit vb.) modüle kaydeder.
7. **`addPassesToEmitFile` :**

8. *Çalışma Mantığı*: Instruction Selection, Register Allocation ve MC (Machine Code) katmanlarını birleştirerek `.o` üretecek backend pass zincirini oluşturur.

4.7 Bölüm Özeti ve Arka Yüz (Backend) Mimarisi Değerlendirmesi

Bu bölümde, LLVM derleyici mimarisinin son aşaması olan **Arka Yüz (Backend) Kod Üretimini** ve **Target Machine** yapılandırmasını inceledik. Soyut LLVM IR'ın somut işlemci komutlarına dönüştürülürken geçtiği Buyruk Seçimi (Instruction Selection), Buyruk Sıralama (Scheduling) ve Yazmaç Tahsisi (Register Allocation) safhalarını öğrendik.

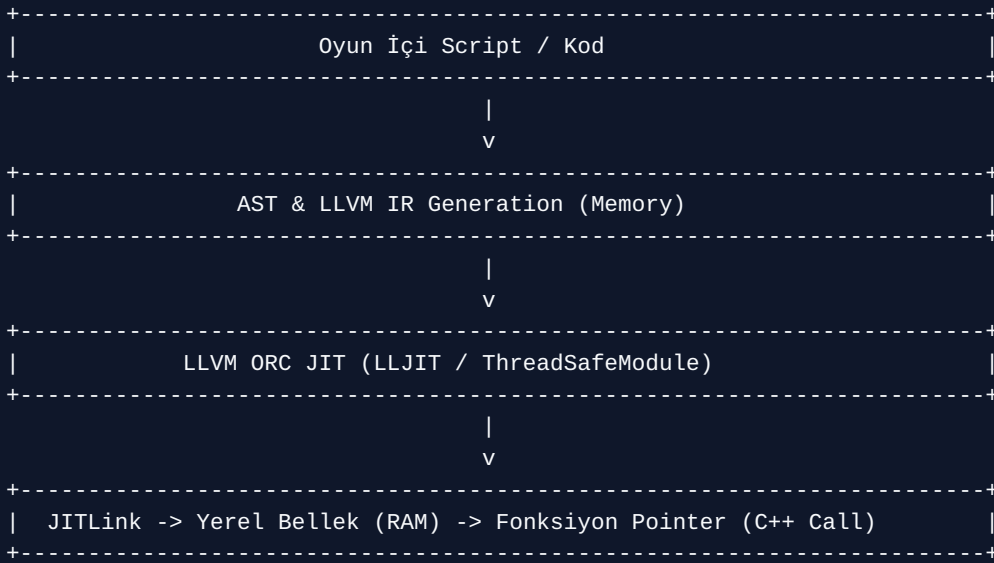
TableGen (`.td`) dilinin işlemci mimarilerini beyan etmedeki rolünü, SelectionDAG ile GlobalSel arasındaki performans ve derleme süresi farklarını ve Machine IR (MIR) seviyesindeki analizi ele aldık. Son olarak C++ TargetMachine API'si vasıtasıyla derlediğimiz oyun dilini disk üzerinde `.o` nesne dosyalarına aktaran yürütme algoritmasını kodladık. Bir sonraki bölümde, oyun motorlarına canlı betik desteği sunan **ORC JIT, MLIR ve Shader Kod Üretimi** konularına geçeceğiz.

Bölüm 5: İleri Düzey Konular: ORC JIT, MLIR, Shader Kod Üretimi ve Derleyici Araçları

5.1 LLVM ORC JIT (On-Request Compilation) Mimarisi ve Oyun İçi Betik (Scripting) Çalıştırma

Oyun motorlarında canlı kod yenileme (Hot-Reloading) ve hızlı betik (scripting) çalıştırma kritik gereksinimlerdir. Geleneksel olarak Lua veya C# gibi diller yorumlanarak (interpreted) veya sanal makine üzerinde çalıştırılır. LLVM **ORC JIT (On-Request Compilation)** API'si sayesinde, oyun içinde yazılan betik kodları çalışma zamanında milisaniyeler içinde doğrudan **özel makine koduna** derlenir ve sıfır performans kaybı ile çalıştırılır.

ORC JIT mimarisinin ana bileşenleri şunlardır: 1. **ExecutionSession**: Tüm JIT durumunu, thread'leri ve sembol tablolarını yönetir. 2. **JITDyLib**: Dinamik kütüphane benzeri sembol konteyneridir. Oyun motorunuzdaki C++ fonksiyonlarını JIT tarafına kaydetmenizi sağlar. 3. **JITLink**: Bellek içi nesne dosyalarını bağlayan ve sayfa izinlerini (Executable/Writable) ayarlayan ultra hızlı bağlayıcıdır.



JIT Scripting Avantajı

Oyun içi düşman yapay zekası (AI) veya fizik hesapları JIT ile derlendiğinde, yorumlanan Lua koduna kıyasla 10 kat ile 50 kat arasında performans artışı elde edilir.

C++ Kodu: Çalışma Zamanında Betik Çalıştıran JIT Motoru (src/jit_engine.cpp)

```

#include <llvm/ExecutionEngine/Orc/LLJIT.h>
#include <llvm/IR/LLVMContext.h>
#include <llvm/IR/Module.h>
#include <llvm/IR/IRBuilder.h>
#include <llvm/Support/InitLLVM.h>
#include <llvm/Support/TargetSelect.h>
#include <llvm/Support/raw_ostream.h>

```

```

#include <iostream>

// Oyun İçi Script Tarafından Çağrılacak C++ Fonksiyonu (Host Engine Function)
extern "C" void GameEngine_LogMessage(const char* msg) {
    std::cout << "[Oyun Motoru JIT Log]: " << msg << std::endl;
}

void runJITScript() {
    // 1. LLVM Target Sistemlerini Başlat
    llvm::InitializeNativeTarget();
    llvm::InitializeNativeTargetAsmPrinter();

    // 2. LLJIT Örneği Oluşturma
    auto jitOrErr = llvm::orc::LLJITBuilder().create();
    if (!jitOrErr) {
        llvm::errs() << "JIT Oluşturulamadı!\n";
        return;
    }
    auto jit = std::move(*jitOrErr);

    // 3. Thread-Safe LLVM Modülü ve Context Yapılandırması
    auto context = std::make_unique<llvm::LLVMContext>();
    auto module = std::make_unique<llvm::Module>("JITScriptModule", *context);
    llvm::IRBuilder<> builder(*context);

    // 4. C++ Tarafındaki Log Message Fonksiyonunu JIT Modülüne Tanımlama
    auto* logFuncType = llvm::FunctionType::get(builder.getVoidTy(), {builder.getPtrTy()}, false);
    auto* logFunc = llvm::Function::Create(logFuncType, llvm::Function::ExternalLinkage, "GameEngine_Log", *module);

    // 5. JIT Script Fonksiyonu Oluşturma: void RunScript()
    auto* scriptFuncType = llvm::FunctionType::get(builder.getVoidTy(), false);
    auto* scriptFunc = llvm::Function::Create(scriptFuncType, llvm::Function::ExternalLinkage, "RunScript", *module);

    auto* bb = llvm::BasicBlock::Create(*context, "entry", scriptFunc);
    builder.SetInsertPoint(bb);

    // Metin Sabiti Oluşturma
    auto* strVal = builder.CreateGlobalString("LLVM ORC JIT Script Basariyla Calisti!", "script_str");
    builder.CreateCall(logFunc, {strVal});
    builder.CreateRetVoid();

    // 6. Modülü JIT'e Ekleme
    auto tsm = llvm::orc::ThreadSafeModule(std::move(module), std::move(context));
    cantFail(jit->addIRModule(std::move(tsm)));

    // 7. JIT İçindeki "RunScript" Sembolünü Bulma ve Çalıştırma
    auto symOrErr = jit->lookup("RunScript");
    if (!symOrErr) {
        llvm::errs() << "JIT Sembolü Bulunamadı!\n";
        return;
    }

    // Sembolü C++ Fonksiyon Göstericisine (Function Pointer) Cast Etme
    void (*scriptFn)() = symOrErr->toPtr<void (*)>();

    // 8. KODU YEREL HIZDA ÇALIŞTIR!
    scriptFn();
}

```

Koda Adım Adım Derinlemesine Bakış ve JIT Yürütme Algoritması

1. **LLJITBuilder().create() :**
2. **Çalışma Mantığı:** Bilgisayarın yerel CPU mimarisini tespit eder, JITLink ve bellek sayfa yöneticisini (Memory Page Manager: Read/Write/Execute izinleri) başlatır.
3. **ThreadSafeModule (TSM) Sarmalayıcısı:**
4. **Çalışma Mantığı:** Module ve LLVMContext nesnelerini kilit mekanizması altında birleştirir. Böylece oyun motorunun arka plan iş parçacıkları (Worker Threads) güvenle derleme yapabilir.
5. **Ev Sahibi Sembol Bağlama (GameEngine_LogMessage):**
6. **Çalışma Mantığı:** C++ tarafında extern "C" olarak tanımlanan fonksiyon, JIT sembol tablosuna kaydedilir. JIT derleyicisi bu sembolün bellek adresini oyun motorunun proses adres alanından (Process Address Space) okur.
7. **symOrErr->toPtr<void (*)()>() Fonksiyon Göstericisi Dönüşümü:**
8. **Çalışma Mantığı:** JIT tarafından RAM'de derlenen makine kodunun başlangıç adresini C++ fonksiyon göstericisine (function pointer) dökümler. scriptFn() çağırısı, araya hiçbir sanal makine veya yorumlayıcı katmanı girmeden doğrudan CPU seviyesinde yürütülür.

5.2 MLIR (Multi-Level Intermediate Representation) ve Oyun Motorları İçin Önemi

LLVM IR harika bir alt seviye temsil olsa da, yüksek seviyeli matematiksel yapılar (matris çarpımları, GPU thread blokları, oyun içi fizik simülasyonları) LLVM IR seviyesine indirildiğinde kaybolur.

MLIR (Multi-Level Intermediate Representation), LLVM projesinin çok seviyeli ara temsil altyapısıdır. **Dialect (Diyalekt)** adı verilen modüler yapıları sayesinde: * **Affine Dialect:** Döngü optimizasyonları ve bellek erişim desenleri için. * **Vector / Linalg Dialect:** Matrix/Vector işlemleri ve SIMD vektörizasyonu için. * **GPU Dialect:** CUDA, ROCm ve Vulkan Compute Kernel yönetimi için.

Oyun dilinizde önce MLIR seviyesinde matris ve vektör optimizasyonları yapabilir, ardından kodu LLVM IR'a indirgeyerek (lowering) donanıma gönderebilirsiniz.

5.3 LLVM IR'dan SPIR-V Üretimi ve DXC (DirectX Shader Compiler)

Modern oyun motorları grafik kartlarında (GPU) paralel hesaplama yapmak için SPIR-V (Vulkan) ve DXIL (DirectX 12) formatlarına ihtiyaç duyar.

- **LLVM SPIR-V Target / Backend:** Oyun dilinizden ürettiğiniz LLVM IR'ı llvm-spirv aracı veya kütüphanesi aracılığıyla doğrudan SPIR-V ikili formatına çevirebilirsiniz. Bu sayede hem CPU hem GPU kodunu **aynı dille** yazabilirsiniz!
- **DXC (DirectX Shader Compiler):** Microsoft'un açık kaynaklı derleyicisidir. LLVM tabanlıdır ve HLSL shader kodlarını SPIR-V veya DXIL formatına dönüştürür.

5.4 Clang Tooling, LLD ve Clang Sanitizer Araçları

Bir oyun programlama dili ekosistemi geliştirmek yalnızca derleyici yazmaktan ibaret değildir; geliştirici araçlarına (Developer Tooling) da ihtiyaç vardır.

1. **Clang Tooling:** Oyun diliniz için otomatik kod biçimlendirici (Formatter), statik analiz araçları (Linter) ve Otomatik Tamamlama (Language Server Protocol - LSP) yazmanızı sağlar.
2. **LLD (LLVM Linker):** Oyun projenizin bağımlılıklarını bağlarken MSVC `link.exe` veya GNU `ld` yerine LLD kullanarak bağlama (linking) sürelerini 5-10 kat kısaltabilirsiniz.
3. **Clang Sanitizer Araçları:** Oyun motorunuzda bellek sızıntılarını ve paralel izlek (multithreading) hatalarını yakalamak için LLVM Sanitize altyapısı entegre edilmelidir:
4. **ASan (AddressSanitizer):** Yığın ve bellek taşmalarını (buffer overflow, use-after-free) yakalar.
5. **TSan (ThreadSanitizer):** Veri yarışlarını (data race) tespit eder.
6. **MSan (MemorySanitizer):** İklendirilmemiş bellek okumalarını yakalar.
7. **UBSan (UndefinedBehaviorSanitizer):** Tanımsız davranışları (sıfıra bölme, tamsayı taşması) tespit eder.

Oyun Testlerinde Sanitizer Kullanımı

Oyun geliştirme esnasında Debug derlemelerini ASan ve TSan ile çalıştırmak, karmaşık oyun fiziği ve çok izlekli (multithreaded) iş dizisi (job system) hatalarını aylar öncesinden tespit etmenizi sağlar.

5.5 Bölüm Özeti ve İleri Düzey Konular Değerlendirmesi

Bu son bölümde, oyun programlama dili geliştirmede çığır açan ileri düzey LLVM teknolojilerini inceledik. **LLVM ORC JIT (LLJIT)** ve **JITLink** mimarisi sayesinde oyun içi betiklerin yorumlama (interpretation) yavaşlığı olmadan, çalışma zamanında yerel C++ hızında nasıl yürütüldüğünü ve C++ host fonksiyonlarının JIT ortamına nasıl bağlandığını kodladık.

Çok seviyeli ara temsil olan **MLIR (Multi-Level Intermediate Representation)** yapısının matris, vektör ve GPU diyalektleri (Dialects) ile yüksek seviyeli oyun fiziği ve matris optimizasyonlarındaki rolünü ele aldık. CPU ve GPU kod bütünlüğü sağlayan **LLVM SPIR-V** ve **DirectX Shader Compiler (DXC)** araçlarını, derleyici araç zincirini tamamlayan **Clang Tooling**, **LLD** bağlayıcısını ve bellek/izlek hatalarını sıfıra indiren **Clang Sanitizer (ASan, TSan, UBSan)** altyapılarını öğrendik. Bu kapsamlı kılavuz ile LLVM kullanarak sıfırdan yüksek performanslı bir oyun programlama dili inşa etmek için gerekli tüm mimari ve pratik bilgiye sahip olduk.